简单学 PyTorch v1.0

初学者必备指南

作者*: jw*

本书按类别整理了 PyTorch 的核心函数。每个函数包含以下内容：

- 代码示例 - 可运行的 Python 代码
- 运行结果 - 实际输出结果
- 说明 - 简明扼要的解释

基于 *PyTorch 2.8.0*

目录

0. 你好世界
1. 张量创建
2. 基本运算
3. 形状操作
4. 索引和切片
5. 归约操作
6. 数学函数
7. 线性代数
8. 神经网络函数
9. 损失函数
10. 池化和卷积
11. 高级操作
12. 自动微分
13. 设备操作
14. 实用工具
15. 比较操作
16. 张量方法

你好世界

本章通过一个简单的例子展示完整的神经网络。让我们教一个小型神经网络学习 XOR 函数。

什么是 **XOR**？

输入 A	输入 B	输出 (A XOR B)
0	0	0
0	1	1
1	0	1
1	1	0

4 个步骤

1. 数据 - 定义输入/输出对
2. 模型 - 创建神经网络
3. 训练 - 从数据学习（前向传播 → 损失 → 反向传播 → 更新）

4. 推理 - 进行预测

完整代码

```
import torch
import torch.nn as nn

# === 你好世界：超简单神经网络 ===
# 目标：学习 XOR 函数 (0^0=0, 0^1=1, 1^0=1, 1^1=0)

# 1. 数据 - 输入/输出对
X = torch.tensor([[0., 0.],
                  [0., 1.],
                  [1., 0.],
                  [1., 1.]])

y = torch.tensor([[0.],
                  [1.],
                  [1.],
                  [0.]])

print("=== 数据 ===")
print(f" 输入 X:\n{X}")
print(f" 答案 y: {y.flatten().tolist()}")

# 2. 模型 - 2 层神经网络
model = nn.Sequential(
    nn.Linear(2, 4),   # 输入 2 个 -> 隐藏 4 个
    nn.ReLU(),         # 激活函数
    nn.Linear(4, 1),   # 隐藏 4 个 -> 输出 1 个
    nn.Sigmoid()       # 输出 0~1
)

print(f"\n=== 模型 ===")
print(model)

# 3. 训练
criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=1.0)

print(f"\n=== 训练 ===")
for epoch in range(1000):
    pred = model(X)
    loss = criterion(pred, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if epoch % 200 == 0:
        print(f" 轮次 {epoch:4d}, 损失: {loss.item():.4f}")

# 4. 推理
print(f"\n=== 推理 ===")
with torch.no_grad():
```

```python
    predictions = model(X)
    rounded = (predictions > 0.5).float()

print(" 输入 -> 预测 -> 四舍五入 -> 答案")
for i in range(4):
    inp = X[i].tolist()
    pred_val = predictions[i].item()
    round_val = int(rounded[i].item())
    target = int(y[i].item())
    status = " 正确" if round_val == target else " 错误"
    print(f"{inp} -> {pred_val:.3f} -> {round_val} -> {target} {status}")

accuracy = (rounded == y).float().mean()
print(f"\n准确率: {accuracy.item()*100:.0f}%")
```

运行结果

```
=== 数据 ===
输入 X:
tensor([[0., 0.],
        [0., 1.],
        [1., 0.],
        [1., 1.]])
答案 y: [0.0, 1.0, 1.0, 0.0]

=== 模型 ===
Sequential(
  (0): Linear(in_features=2, out_features=4, bias=True)
  (1): ReLU()
  (2): Linear(in_features=4, out_features=1, bias=True)
  (3): Sigmoid()
)

=== 训练 ===
轮次    0, 损失: 0.2455
轮次  200, 损失: 0.1671
轮次  400, 损失: 0.1668
轮次  600, 损失: 0.1668
轮次  800, 损失: 0.1667

=== 推理 ===
输入 -> 预测 -> 四舍五入 -> 答案
[0.0, 0.0] -> 0.334 -> 0 -> 0 正确
[0.0, 1.0] -> 0.988 -> 1 -> 1 正确
[1.0, 0.0] -> 0.334 -> 0 -> 1 错误
[1.0, 1.0] -> 0.334 -> 0 -> 0 正确

准确率: 75%
```

核心概念

- nn.Sequential - 按顺序堆叠层
- nn.Linear(in, out) - 全连接层

- nn.ReLU() - 激活函数
- nn.MSELoss() - 损失函数（误差多少？)
- optimizer.zero_grad() - 初始化梯度
- loss.backward() - 计算梯度（反向传播）
- optimizer.step() - 更新权重
- torch.no_grad() - 推理时禁用梯度

完成！你完成了第一个神经网络训练。

张量创建

torch.tensor(data) 从数据（列表、*numpy* 数组等）创建张量。自动推断 *dtype*。

示例:

```
import torch
# Create tensor from Python list
x = torch.tensor([[1, 2], [3, 4]])
print("Tensor from list:")
print(x)
print(f"Shape: {x.shape}, dtype: {x.dtype}")
```

运行结果:

```
Tensor from list:
tensor([[1, 2],
        [3, 4]])
Shape: torch.Size([2, 2]), dtype: torch.int64
```

torch.zeros(*size) 创建全零张量。常用于初始化。

示例:

```
import torch
# Create tensor filled with zeros
x = torch.zeros(2, 3)
print("Zeros tensor (2x3):")
print(x)
```

运行结果:

```
Zeros tensor (2x3):
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

torch.ones(*size) 创建全 *1* 张量。常用于掩码或初始化。

示例:

```
import torch
# Create tensor filled with ones
x = torch.ones(3, 2)
print("Ones tensor (3x2):")
print(x)
```

运行结果:

```
Ones tensor (3x2):
tensor([[1., 1.],
        [1., 1.],
        [1., 1.]])
```

torch.eye(n) # identity matrix 创建单位矩阵（对角线为 *1*，其他为 *0*）。用于线性代数。

示例:

```python
import torch
# Create identity matrix
x = torch.eye(3)
print("Identity matrix (3x3):")
print(x)
```

运行结果:

```
Identity matrix (3x3):
tensor([[1., 0., 0.],
        [0., 1., 0.],
        [0., 0., 1.]])
```

torch.arange(start, end, step) 创建等间隔的 *1D* 张量。类似 *Python* 的 *range()*。

示例:

```python
import torch
# Create range tensor
x = torch.arange(0, 10, 2)
print("Range [0, 10) step 2:")
print(x)
```

运行结果:

```
Range [0, 10) step 2:
tensor([0, 2, 4, 6, 8])
```

torch.linspace(start, end, steps) 创建在起点和终点之间均匀分布的张量。

示例:

```python
import torch
# Create linearly spaced tensor
x = torch.linspace(0, 1, 5)
print("Linspace 0 to 1, 5 points:")
print(x)
```

运行结果:

```
Linspace 0 to 1, 5 points:
tensor([0.0000, 0.2500, 0.5000, 0.7500, 1.0000])
```

torch.logspace(start, end, steps) 创建对数尺度均匀分布的张量。适合设置学习率。

示例:

```python
import torch
# Create logarithmically spaced tensor
```

```
x = torch.logspace(0, 2, 3)  # 10^0, 10^1, 10^2
print("Logspace 10^0 to 10^2:")
print(x)
```

运行结果:

```
Logspace 10^0 to 10^2:
tensor([  1.,  10., 100.])
```

torch.rand(*size) # uniform [0,1) 创建 *0* 到 *1* 之间均匀分布的随机张量。

示例:

```
import torch
torch.manual_seed(42)
# Create random tensor [0, 1)
x = torch.rand(2, 3)
print("Random uniform [0,1):")
print(x)
```

运行结果:

```
Random uniform [0,1):
tensor([[0.8823, 0.9150, 0.3829],
        [0.9593, 0.3904, 0.6009]])
```

torch.randn(*size) # normal N(0,1) 从标准正态分布（均值 *=0*，标准差 *=1*）采样创建张量。

示例:

```
import torch
torch.manual_seed(42)
# Create random tensor from normal distribution
x = torch.randn(2, 3)
print("Random normal N(0,1):")
print(x)
```

运行结果:

```
Random normal N(0,1):
tensor([[ 0.3367,  0.1288,  0.2345],
        [ 0.2303, -1.1229, -0.1863]])
```

torch.randint(low, high, size) 创建 *[low, high)* 范围内的随机整数张量。

示例:

```
import torch
torch.manual_seed(42)
# Create random integers
x = torch.randint(0, 10, (2, 3))
print("Random integers [0, 10):")
print(x)
```

运行结果:

```
Random integers [0, 10):
tensor([[2, 7, 6],
```

```
        [4, 6, 5]])
```

torch.empty(*size) 创建未初始化的张量。比 *zeros/ones* 快但包含垃圾值。

示例:

```python
import torch
# Create uninitialized tensor
x = torch.empty(2, 2)
print("Empty tensor (uninitialized):")
print(x)
print("Warning: Contains garbage values!")
```

运行结果:

```
Empty tensor (uninitialized):
tensor([[0., 0.],
        [0., 0.]])
Warning: Contains garbage values!
```

torch.full(size, fill_value) 创建填充特定值的张量。适合常量。

示例:

```python
import torch
# Create tensor filled with specific value
x = torch.full((2, 3), 7.0)
print("Tensor filled with 7.0:")
print(x)
```

运行结果:

```
Tensor filled with 7.0:
tensor([[7., 7., 7.],
        [7., 7., 7.]])
```

torch.zeros_like(x), ones_like(x) 创建与输入张量相同形状和 *dtype* 的 *0/1* 张量。

示例:

```python
import torch
original = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
zeros = torch.zeros_like(original)
ones = torch.ones_like(original)
print("Original:", original.shape)
print("Zeros like:", zeros)
print("Ones like:", ones)
```

运行结果:

```
Original: torch.Size([2, 2])
Zeros like: tensor([[0., 0.],
        [0., 0.]])
Ones like: tensor([[1., 1.],
        [1., 1.]])
```

基本运算

`torch.add(a, b) or a + b` 逐元素加法。支持不同形状的广播。

示例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
result = torch.add(a, b)
print(f"{a} + {b} = {result}")
```

运行结果:

```
tensor([1, 2, 3]) + tensor([4, 5, 6]) = tensor([5, 7, 9])
```

`torch.sub(a, b) or a - b` 逐元素减法。支持广播。

示例:

```
import torch
a = torch.tensor([5, 6, 7])
b = torch.tensor([1, 2, 3])
result = torch.sub(a, b)
print(f"{a} - {b} = {result}")
```

运行结果:

```
tensor([5, 6, 7]) - tensor([1, 2, 3]) = tensor([4, 4, 4])
```

`torch.mul(a, b) or a * b` 逐元素乘法 (*Hadamard* 积)。不是矩阵乘法。

示例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
result = torch.mul(a, b)
print(f"{a} * {b} = {result}")
```

运行结果:

```
tensor([1, 2, 3]) * tensor([4, 5, 6]) = tensor([ 4, 10, 18])
```

`torch.div(a, b) or a / b` 逐元素除法。使用 *float* 张量避免整数除法。

示例:

```
import torch
a = torch.tensor([10.0, 20.0, 30.0])
b = torch.tensor([2.0, 4.0, 5.0])
result = torch.div(a, b)
print(f"{a} / {b} = {result}")
```

运行结果:

```
tensor([10., 20., 30.]) / tensor([2., 4., 5.]) = tensor([5., 5., 6.])
```

torch.matmul(a, b) or a @ b 矩阵乘法。对于 *2D* 张量，执行标准矩阵乘积。

示例:

```
import torch
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6], [7, 8]])
result = torch.matmul(a, b)
print("Matrix A:")
print(a)
print("Matrix B:")
print(b)
print("A @ B:")
print(result)
```

运行结果:

```
Matrix A:
tensor([[1, 2],
        [3, 4]])
Matrix B:
tensor([[5, 6],
        [7, 8]])
A @ B:
tensor([[19, 22],
        [43, 50]])
```

torch.pow(a, exp) or a ** exp 逐元素幂运算。可使用标量或张量指数。

示例:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
result = torch.pow(x, 2)
print(f"{x} ** 2 = {result}")
```

运行结果:

```
tensor([1., 2., 3.]) ** 2 = tensor([1., 4., 9.])
```

torch.abs(x) # absolute value 返回每个元素的绝对值。

示例:

```
import torch
x = torch.tensor([-1, -2, 3, -4])
result = torch.abs(x)
print(f"abs({x}) = {result}")
```

运行结果:

```
abs(tensor([-1, -2,  3, -4])) = tensor([1, 2, 3, 4])
```

torch.neg(x) # negative 返回每个元素的负数。等同于-*x*。

示例:

```
import torch
x = torch.tensor([1, -2, 3])
result = torch.neg(x)
print(f"neg({x}) = {result}")
```

运行结果:

```
neg(tensor([ 1, -2,  3])) = tensor([-1,  2, -3])
```

`torch.reciprocal(x)  # 1/x` 返回每个元素的倒数（*1/x*）。

示例:

```
import torch
x = torch.tensor([1.0, 2.0, 4.0])
result = torch.reciprocal(x)
print(f"1/{x} = {result}")
```

运行结果:

```
1/tensor([1., 2., 4.]) = tensor([1.0000, 0.5000, 0.2500])
```

`torch.remainder(a, b)  # remainder` 逐元素取余（模运算）。

示例:

```
import torch
a = torch.tensor([10, 11, 12])
b = torch.tensor([3, 3, 3])
result = torch.remainder(a, b)
print(f"{a} % {b} = {result}")
```

运行结果:

```
tensor([10, 11, 12]) % tensor([3, 3, 3]) = tensor([1, 2, 0])
```

形状操作

`x.reshape(*shape)` 返回新形状的张量。元素总数必须相同。可能复制数据。

示例:

```
import torch
x = torch.arange(6)
print(f"Original: {x}")
reshaped = x.reshape(2, 3)
print("Reshaped to (2, 3):")
print(reshaped)
```

运行结果:

```
Original: tensor([0, 1, 2, 3, 4, 5])
Reshaped to (2, 3):
tensor([[0, 1, 2],
        [3, 4, 5]])
```

x.view(*shape) 返回新形状的视图。需要连续内存。共享数据。

示例:

```
import torch
x = torch.arange(12)
print(f"Original: {x}")
viewed = x.view(3, 4)
print("View as (3, 4):")
print(viewed)
```

运行结果:

```
Original: tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
View as (3, 4):
tensor([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11]])
```

x.transpose(dim0, dim1) 交换两个维度。对于 *2D* 等同于矩阵转置。

示例:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Original (2x3):")
print(x)
transposed = x.transpose(0, 1)
print("Transposed (3x2):")
print(transposed)
```

运行结果:

```
Original (2x3):
tensor([[1, 2, 3],
        [4, 5, 6]])
Transposed (3x2):
tensor([[1, 4],
        [2, 5],
        [3, 6]])
```

x.permute(*dims) 重新排列所有维度。比 *transpose* 更灵活。

示例:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Original shape: {x.shape}")
permuted = x.permute(2, 0, 1)
print(f"Permuted shape: {permuted.shape}")
```

运行结果:

```
Original shape: torch.Size([2, 3, 4])
Permuted shape: torch.Size([4, 2, 3])
```

x.squeeze(dim) 移除大小为 *1* 的维度。降低张量的秩。

示例:

```python
import torch
x = torch.zeros(1, 3, 1, 4)
print(f"Original shape: {x.shape}")
squeezed = x.squeeze()
print(f"Squeezed shape: {squeezed.shape}")
```

运行结果:

```
Original shape: torch.Size([1, 3, 1, 4])
Squeezed shape: torch.Size([3, 4])
```

x.unsqueeze(dim) 在指定位置添加大小为 *1* 的维度。

示例:

```python
import torch
x = torch.tensor([1, 2, 3])
print(f"Original shape: {x.shape}")
unsqueezed = x.unsqueeze(0)
print(f"Unsqueezed at dim 0: {unsqueezed.shape}")
print(unsqueezed)
```

运行结果:

```
Original shape: torch.Size([3])
Unsqueezed at dim 0: torch.Size([1, 3])
tensor([[1, 2, 3]])
```

x.flatten(start_dim, end_dim) 将张量展平为 *1D*。可选起始/结束维度用于部分展平。

示例:

```python
import torch
x = torch.randn(2, 3, 4)
print(f"Original shape: {x.shape}")
flat = x.flatten()
print(f"Flattened: {flat.shape}")
```

运行结果:

```
Original shape: torch.Size([2, 3, 4])
Flattened: torch.Size([24])
```

x.expand(*sizes) 沿大小为 *1* 的维度重复扩展张量。不复制数据。

示例:

```python
import torch
x = torch.tensor([[1], [2], [3]])
print(f"Original: {x.shape}")
expanded = x.expand(3, 4)
print("Expanded to (3, 4):")
print(expanded)
```

运行结果:

```
Original: torch.Size([3, 1])
Expanded to (3, 4):
```

```
tensor([[1, 1, 1, 1],
        [2, 2, 2, 2],
        [3, 3, 3, 3]])
```

x.repeat(*sizes) 沿每个维度重复张量。创建新内存。

示例:

```
import torch
x = torch.tensor([1, 2])
print(f"Original: {x}")
repeated = x.repeat(3)
print(f"Repeated 3x: {repeated}")
```

运行结果:

```
Original: tensor([1, 2])
Repeated 3x: tensor([1, 2, 1, 2, 1, 2])
```

x.contiguous() 返回内存连续的张量。*transpose* 后在 *view()* 前需要。

示例:

```
import torch
x = torch.randn(3, 4)
y = x.transpose(0, 1)
print(f"Is contiguous: {y.is_contiguous()}")
z = y.contiguous()
print(f"After contiguous(): {z.is_contiguous()}")
```

运行结果:

```
Is contiguous: False
After contiguous(): True
```

索引和切片

x[i] # index 基本索引，返回索引 *i* 处的行/元素。

示例:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Tensor:")
print(x)
print(f"x[0] = {x[0]}")
print(f"x[1] = {x[1]}")
```

运行结果:

```
Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
x[0] = tensor([1, 2, 3])
x[1] = tensor([4, 5, 6])
```

`x[i:j]  # slice` 使用 *start:stop:step* 切片。类似 *Python* 列表。

示例:

```
import torch
x = torch.arange(10)
print(f"Original: {x}")
print(f"x[2:5] = {x[2:5]}")
print(f"x[::2] = {x[::2]}")
```

运行结果:

```
Original: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
x[2:5] = tensor([2, 3, 4])
x[::2] = tensor([0, 2, 4, 6, 8])
```

`x[..., i]  # ellipsis` 省略号 *(...)* 代表所有剩余维度。

示例:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Shape: {x.shape}")
print(f"x[..., 0] shape: {x[..., 0].shape}")
print(f"x[0, ...] shape: {x[0, ...].shape}")
```

运行结果:

```
Shape: torch.Size([2, 3, 4])
x[..., 0] shape: torch.Size([2, 3])
x[0, ...] shape: torch.Size([3, 4])
```

`x[:, -1]  # last column` 负索引从末尾计数。*-1* 是最后一个元素。

示例:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Tensor:")
print(x)
print(f"Last column x[:, -1] = {x[:, -1]}")
```

运行结果:

```
Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
Last column x[:, -1] = tensor([3, 6])
```

`torch.index_select(x, dim, idx)` 使用索引张量沿维度选择元素。

示例:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
idx = torch.tensor([0, 2])
result = torch.index_select(x, 0, idx)
print("Original:")
print(x)
```

```
print(f"Select rows {idx.tolist()}:")
print(result)
```

运行结果:

```
Original:
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
Select rows [0, 2]:
tensor([[1, 2, 3],
        [7, 8, 9]])
```

`torch.masked_select(x, mask)` 返回掩码为 *True* 处元素的 *1D* 张量。

示例:

```
import torch
x = torch.tensor([[1, 2], [3, 4]])
mask = x > 2
print(f"Tensor: {x}")
print(f"Mask (>2): {mask}")
print(f"Selected: {torch.masked_select(x, mask)}")
```

运行结果:

```
Tensor: tensor([[1, 2],
        [3, 4]])
Mask (>2): tensor([[False, False],
        [ True,  True]])
Selected: tensor([3, 4])
```

`torch.gather(x, dim, idx)  # gather` 按索引沿轴收集值。适合从分布中选择。

示例:

```
import torch
x = torch.tensor([[1, 2], [3, 4]])
idx = torch.tensor([[0, 0], [1, 0]])
result = torch.gather(x, 1, idx)
print("Original:")
print(x)
print(f"Gather with indices {idx.tolist()}:")
print(result)
```

运行结果:

```
Original:
tensor([[1, 2],
        [3, 4]])
Gather with indices [[0, 0], [1, 0]]:
tensor([[1, 1],
        [4, 3]])
```

`torch.scatter(x, dim, idx, src)` 将 *src* 的值写入 *x* 的 *idx* 指定位置。*gather* 的逆操作。

示例:

```python
import torch
x = torch.zeros(3, 5)
idx = torch.tensor([[0, 1, 2, 0, 0], [2, 0, 0, 1, 2]])
src = torch.ones(2, 5)
result = x.scatter(0, idx, src)
print("Scatter result:")
print(result)
```

运行结果:

```
Scatter result:
tensor([[1., 1., 1., 1., 1.],
        [0., 1., 0., 1., 0.],
        [1., 0., 1., 0., 1.]])
```

`torch.where(cond, x, y)  # conditional` 条件为 *True* 处返回 *x* 的元素，否则返回 *y* 的元素。

示例:

```python
import torch
x = torch.tensor([1, 2, 3, 4, 5])
y = torch.tensor([10, 20, 30, 40, 50])
cond = x > 3
result = torch.where(cond, x, y)
print(f"x: {x}")
print(f"y: {y}")
print(f"where(x>3, x, y): {result}")
```

运行结果:

```
x: tensor([1, 2, 3, 4, 5])
y: tensor([10, 20, 30, 40, 50])
where(x>3, x, y): tensor([10, 20, 30,  4,  5])
```

`torch.take(x, indices)  # flat index` 将张量视为 *1D*，返回指定索引的元素。

示例:

```python
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
idx = torch.tensor([0, 2, 5])
result = torch.take(x, idx)
print(f"Tensor (flattened would be {x.flatten().tolist()})")
print(f"Take indices {idx.tolist()}: {result}")
```

运行结果:

```
Tensor (flattened would be [1, 2, 3, 4, 5, 6])
Take indices [0, 2, 5]: tensor([1, 3, 6])
```

Σ 归约操作

`x.sum(dim, keepdim)` 求和元素。可选 *dim* 指定轴。*keepdim* 保持维度。

示例:

```python
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Tensor:")
print(x)
print(f"Sum all: {x.sum()}")
print(f"Sum dim=0: {x.sum(dim=0)}")
print(f"Sum dim=1: {x.sum(dim=1)}")
```

运行结果:

```
Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
Sum all: 21
Sum dim=0: tensor([5, 7, 9])
Sum dim=1: tensor([ 6, 15])
```

`x.mean(dim, keepdim)` 计算均值。需要 *float* 张量。

示例:

```python
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
print("Tensor:")
print(x)
print(f"Mean all: {x.mean()}")
print(f"Mean dim=1: {x.mean(dim=1)}")
```

运行结果:

```
Tensor:
tensor([[1., 2.],
        [3., 4.]])
Mean all: 2.5
Mean dim=1: tensor([1.5000, 3.5000])
```

`x.std(dim, unbiased)` 计算标准差。*unbiased=True* 使用 *N-1* 作为分母。

示例:

```python
import torch
x = torch.tensor([1.0, 2.0, 3.0, 4.0, 5.0])
print(f"Data: {x}")
print(f"Std (unbiased): {x.std():.4f}")
print(f"Std (biased): {x.std(unbiased=False):.4f}")
```

运行结果:

```
Data: tensor([1., 2., 3., 4., 5.])
Std (unbiased): 1.5811
Std (biased): 1.4142
```

`x.var(dim, unbiased)` 计算方差（标准差的平方）。

示例:

```python
import torch
x = torch.tensor([1.0, 2.0, 3.0, 4.0, 5.0])
print(f"Data: {x}")
print(f"Variance: {x.var():.4f}")
```

运行结果:

```
Data: tensor([1., 2., 3., 4., 5.])
Variance: 2.5000
```

`x.max(dim)  # values & indices` 返回沿维度的最大值和索引。

示例:

```python
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
vals, idxs = x.max(dim=1)
print(f"Max per row: values={vals}, indices={idxs}")
```

运行结果:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Max per row: values=tensor([5, 6]), indices=tensor([1, 2])
```

`x.min(dim)  # values & indices` 返回沿维度的最小值和索引。

示例:

```python
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
vals, idxs = x.min(dim=1)
print(f"Min per row: values={vals}, indices={idxs}")
```

运行结果:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Min per row: values=tensor([1, 2]), indices=tensor([0, 1])
```

`x.argmax(dim)  # indices only` 返回最大值的索引。常与 *softmax* 输出一起使用。

示例:

```python
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
print(f"Argmax (all): {x.argmax()}")
print(f"Argmax dim=1: {x.argmax(dim=1)}")
```

运行结果:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Argmax (all): 5
Argmax dim=1: tensor([1, 2])
```

x.argmin(dim) # indices only 返回最小值的索引。

示例:

```python
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
print(f"Argmin dim=1: {x.argmin(dim=1)}")
```

运行结果:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Argmin dim=1: tensor([0, 1])
```

x.median(dim) 返回沿维度的中位数和索引。

示例:

```python
import torch
x = torch.tensor([[1, 5, 3], [4, 2, 6]])
print("Tensor:")
print(x)
vals, idxs = x.median(dim=1)
print(f"Median per row: values={vals}, indices={idxs}")
```

运行结果:

```
Tensor:
tensor([[1, 5, 3],
        [4, 2, 6]])
Median per row: values=tensor([3, 4]), indices=tensor([2, 0])
```

x.mode(dim) 返回沿维度的众数。

示例:

```python
import torch
x = torch.tensor([[1, 1, 2], [3, 3, 3]])
print("Tensor:")
print(x)
vals, idxs = x.mode(dim=1)
print(f"Mode per row: values={vals}")
```

运行结果:

```
Tensor:
tensor([[1, 1, 2],
        [3, 3, 3]])
Mode per row: values=tensor([1, 3])
```

x.prod(dim) # product 计算沿维度的元素乘积。

示例:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Tensor:")
print(x)
print(f"Product all: {x.prod()}")
print(f"Product dim=1: {x.prod(dim=1)}")
```

运行结果:

```
Tensor:
tensor([[1, 2, 3],
        [4, 5, 6]])
Product all: 720
Product dim=1: tensor([  6, 120])
```

x.cumsum(dim) # cumulative sum 计算沿维度的累积和。每个元素是之前所有元素的和。

示例:

```
import torch
x = torch.tensor([1, 2, 3, 4])
print(f"Original: {x}")
print(f"Cumsum: {x.cumsum(dim=0)}")
```

运行结果:

```
Original: tensor([1, 2, 3, 4])
Cumsum: tensor([ 1,  3,  6, 10])
```

x.norm(p, dim) # Lp norm 计算 *Lp* 范数。*L2*（欧几里得）是默认值。*L1* 是曼哈顿距离。

示例:

```
import torch
x = torch.tensor([3.0, 4.0])
print(f"Vector: {x}")
print(f"L2 norm: {x.norm():.4f}")  # sqrt(9+16) = 5
print(f"L1 norm: {x.norm(p=1):.4f}")  # 3+4 = 7
```

运行结果:

```
Vector: tensor([3., 4.])
L2 norm: 5.0000
L1 norm: 7.0000
```

∫ 数学函数

torch.sin(x), cos(x), tan(x) 三角函数。输入为弧度。

示例:

```
import torch
import math
x = torch.tensor([0, math.pi/2, math.pi])
```

```
print(f"x: {x}")
print(f"sin(x): {torch.sin(x)}")
print(f"cos(x): {torch.cos(x)}")
```

运行结果:

```
x: tensor([0.0000, 1.5708, 3.1416])
sin(x): tensor([ 0.0000e+00,  1.0000e+00, -8.7423e-08])
cos(x): tensor([ 1.0000e+00, -4.3711e-08, -1.0000e+00])
```

torch.asin(x), acos(x), atan(x) 反三角函数。返回弧度。

示例:

```
import torch
x = torch.tensor([0.0, 0.5, 1.0])
print(f"x: {x}")
print(f"asin(x): {torch.asin(x)}")
print(f"acos(x): {torch.acos(x)}")
```

运行结果:

```
x: tensor([0.0000, 0.5000, 1.0000])
asin(x): tensor([0.0000, 0.5236, 1.5708])
acos(x): tensor([1.5708, 1.0472, 0.0000])
```

torch.sinh(x), cosh(x), tanh(x) 双曲函数。*tanh* 常用作激活函数。

示例:

```
import torch
x = torch.tensor([-1.0, 0.0, 1.0])
print(f"x: {x}")
print(f"tanh(x): {torch.tanh(x)}")
```

运行结果:

```
x: tensor([-1.,  0.,  1.])
tanh(x): tensor([-0.7616,  0.0000,  0.7616])
```

torch.exp(x), log(x), log10(x) 指数和对数函数。*log* 是自然对数（底数 *e*）。

示例:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
print(f"x: {x}")
print(f"exp(x): {torch.exp(x)}")
print(f"log(exp(x)): {torch.log(torch.exp(x))}")
```

运行结果:

```
x: tensor([1., 2., 3.])
exp(x): tensor([ 2.7183,  7.3891, 20.0855])
log(exp(x)): tensor([1.0000, 2.0000, 3.0000])
```

`torch.sqrt(x), rsqrt(x)` 平方根和逆平方根（*1/sqrt(x)*）。

示例:

```
import torch
x = torch.tensor([1.0, 4.0, 9.0])
print(f"x: {x}")
print(f"sqrt(x): {torch.sqrt(x)}")
print(f"rsqrt(x): {torch.rsqrt(x)}")  # 1/sqrt(x)
```

运行结果:

```
x: tensor([1., 4., 9.])
sqrt(x): tensor([1., 2., 3.])
rsqrt(x): tensor([1.0000, 0.5000, 0.3333])
```

`torch.floor(x), ceil(x), round(x)` 舍入函数。*floor* 向下，*ceil* 向上。

示例:

```
import torch
x = torch.tensor([1.2, 2.5, 3.7])
print(f"x: {x}")
print(f"floor(x): {torch.floor(x)}")
print(f"ceil(x): {torch.ceil(x)}")
print(f"round(x): {torch.round(x)}")
```

运行结果:

```
x: tensor([1.2000, 2.5000, 3.7000])
floor(x): tensor([1., 2., 3.])
ceil(x): tensor([2., 3., 4.])
round(x): tensor([1., 2., 4.])
```

`torch.clamp(x, min, max)` 将值限制在 *[min, max]* 范围内。超出范围的值设为边界值。

示例:

```
import torch
x = torch.tensor([-2, 0, 3, 5, 10])
result = torch.clamp(x, min=0, max=5)
print(f"Original: {x}")
print(f"Clamped [0,5]: {result}")
```

运行结果:

```
Original: tensor([-2,  0,  3,  5, 10])
Clamped [0,5]: tensor([0, 0, 3, 5, 5])
```

`torch.sign(x)` 根据每个元素的符号返回*-1*、*0* 或 *1*。

示例:

```
import torch
x = torch.tensor([-3, 0, 5])
print(f"x: {x}")
print(f"sign(x): {torch.sign(x)}")
```

运行结果:

```
x: tensor([-3,  0,  5])
sign(x): tensor([-1,  0,  1])
```

torch.sigmoid(x) *Sigmoid* 函数：*1/(1+e^-x)*。将值映射到 *(0, 1)*。用于二分类。

示例:

```python
import torch
x = torch.tensor([-2.0, 0.0, 2.0])
print(f"x: {x}")
print(f"sigmoid(x): {torch.sigmoid(x)}")
```

运行结果:

```
x: tensor([-2.,  0.,  2.])
sigmoid(x): tensor([0.1192, 0.5000, 0.8808])
```

线性代数

torch.mm(a, b) # 2D matrix mult *2D* 张量的矩阵乘法。对于批量或高维使用 *matmul*。

示例:

```python
import torch
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6], [7, 8]])
result = torch.mm(a, b)
print("A @ B =")
print(result)
```

运行结果:

```
A @ B =
tensor([[19, 22],
        [43, 50]])
```

torch.bmm(a, b) # batch mm 批量矩阵乘法。第一个维度是批量大小。

示例:

```python
import torch
a = torch.randn(10, 3, 4)  # batch of 10 matrices
b = torch.randn(10, 4, 5)
result = torch.bmm(a, b)
print(f"Batch shapes: {a.shape} @ {b.shape} = {result.shape}")
```

运行结果:

```
Batch shapes: torch.Size([10, 3, 4]) @ torch.Size([10, 4, 5]) = torch.Size([10, 3, 5])
```

torch.mv(mat, vec) # matrix-vector 矩阵-向量乘法。*vec* 被视为列向量。

示例:

```python
import torch
mat = torch.tensor([[1, 2], [3, 4]])
vec = torch.tensor([1, 1])
```

```
result = torch.mv(mat, vec)
print(f"Matrix @ vector = {result}")
```

运行结果:

```
Matrix @ vector = tensor([3, 7])
```

`torch.dot(a, b)  # 1D dot product` *1D* 张量的点积。逐元素乘积之和。

示例:

```
import torch
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])
result = torch.dot(a, b)
print(f"{a} . {b} = {result}")
```

运行结果:

```
tensor([1., 2., 3.]) . tensor([4., 5., 6.]) = 32.0
```

`torch.det(x)  # determinant` 计算方阵的行列式。

示例:

```
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
det = torch.det(x)
print("Matrix:")
print(x)
print(f"Determinant: {det:.4f}")
```

运行结果:

```
Matrix:
tensor([[1., 2.],
        [3., 4.]])
Determinant: -2.0000
```

`torch.inverse(x)  # matrix inverse` 计算矩阵的逆。*A @ A^-1* = 单位矩阵。

示例:

```
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
inv = torch.inverse(x)
print("Original:")
print(x)
print("Inverse:")
print(inv)
print("X @ X^-1:")
print(torch.mm(x, inv))
```

运行结果:

```
Original:
tensor([[1., 2.],
        [3., 4.]])
```

```
Inverse:
tensor([[-2.0000,  1.0000],
        [ 1.5000, -0.5000]])
X @ X^-1:
tensor([[1., 0.],
        [0., 1.]])
```

`torch.svd(x)  # singular value decomp` 奇异值分解。*X = U @ diag(S) @ V^T*

示例:

```
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])
U, S, V = torch.svd(x)
print(f"Original shape: {x.shape}")
print(f"U: {U.shape}, S: {S.shape}, V: {V.shape}")
print(f"Singular values: {S}")
```

运行结果:

```
Original shape: torch.Size([3, 2])
U: torch.Size([3, 2]), S: torch.Size([2]), V: torch.Size([2, 2])
Singular values: tensor([9.5255, 0.5143])
```

`torch.eig(x)  # eigenvalues` 计算特征值。一般用 *linalg.eig*，对称用 *linalg.eigvalsh*。

示例:

```
import torch
x = torch.tensor([[1.0, 2.0], [2.0, 1.0]])
eigenvalues = torch.linalg.eigvalsh(x)  # For symmetric matrices
print("Matrix:")
print(x)
print(f"Eigenvalues: {eigenvalues}")
```

运行结果:

```
Matrix:
tensor([[1., 2.],
        [2., 1.]])
Eigenvalues: tensor([-1.,  3.])
```

`torch.linalg.norm(x, ord)` 计算矩阵或向量范数。*Frobenius* 是矩阵的默认值。

示例:

```
import torch
x = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
print("Matrix:")
print(x)
print(f"Frobenius norm: {torch.linalg.norm(x):.4f}")
print(f"L1 norm: {torch.linalg.norm(x, ord=1):.4f}")
```

运行结果:

```
Matrix:
tensor([[1., 2.],
        [3., 4.]])
```

```
Frobenius norm: 5.4772
L1 norm: 6.0000
```

`torch.linalg.solve(A, b)` 求解线性方程组 $Ax = b$。比求逆矩阵更稳定。

示例:

```
import torch
A = torch.tensor([[3.0, 1.0], [1.0, 2.0]])
b = torch.tensor([9.0, 8.0])
x = torch.linalg.solve(A, b)
print(f"Solving Ax = b")
print(f"x = {x}")
print(f"Verify A@x = {A @ x}")
```

运行结果:

```
Solving Ax = b
x = tensor([2., 3.])
Verify A@x = tensor([9., 8.])
```

`torch.trace(x)  # sum of diagonal` 对角元素之和。单位矩阵的 *trace* 是维数。

示例:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
print("Matrix:")
print(x)
print(f"Trace: {torch.trace(x)}")  # 1+5+9 = 15
```

运行结果:

```
Matrix:
tensor([[1, 2, 3],
        [4, 5, 6],
        [7, 8, 9]])
Trace: 15
```

`torch.outer(a, b)  # outer product` 两个向量的外积。结果形状为 *(len(a), len(b))*。

示例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5])
result = torch.outer(a, b)
print(f"a: {a}, b: {b}")
print("Outer product:")
print(result)
```

运行结果:

```
a: tensor([1, 2, 3]), b: tensor([4, 5])
Outer product:
tensor([[ 4,  5],
        [ 8, 10],
        [12, 15]])
```

神经网络函数

激活函数为神经网络添加非线性。

F.relu(x) # max(0, x) *ReLU* (整流线性单元)。返回 *max(0, x)*。最常用的激活函数。

示例:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])
print(f"Input: {x}")
print(f"ReLU:  {F.relu(x)}")
```

运行结果:

```
Input: tensor([-2., -1.,  0.,  1.,  2.])
ReLU:  tensor([0., 0., 0., 1., 2.])
```

F.leaky_relu(x, neg_slope) 类似 *ReLU* 但允许小的负值。防止'*dying ReLU*'问题。

示例:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])
print(f"Input: {x}")
print(f"Leaky ReLU: {F.leaky_relu(x, 0.1)}")
```

运行结果:

```
Input: tensor([-2., -1.,  0.,  1.,  2.])
Leaky ReLU: tensor([-0.2000, -0.1000,  0.0000,  1.0000,  2.0000])
```

F.gelu(x) # Gaussian Error 高斯误差线性单元。用于 *Transformer* (*BERT*、*GPT*)。

示例:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, -1.0, 0.0, 1.0, 2.0])
print(f"Input: {x}")
print(f"GELU:  {F.gelu(x)}")
```

运行结果:

```
Input: tensor([-2., -1.,  0.,  1.,  2.])
GELU:  tensor([-0.0455, -0.1587,  0.0000,  0.8413,  1.9545])
```

F.sigmoid(x) # 1/(1+e^-x) 将值映射到 *(0, 1)*。用于二分类输出。

示例:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, 0.0, 2.0])
```

```
print(f"Input: {x}")
print(f"Sigmoid: {F.sigmoid(x)}")
```

运行结果:

```
Input: tensor([-2.,  0.,  2.])
Sigmoid: tensor([0.1192, 0.5000, 0.8808])
```

`F.tanh(x)  # hyperbolic tan` 将值映射到 *(-1, 1)*。零中心，比 *sigmoid* 更适合隐藏层。

示例:

```
import torch
import torch.nn.functional as F
x = torch.tensor([-2.0, 0.0, 2.0])
print(f"Input: {x}")
print(f"Tanh: {F.tanh(x)}")
```

运行结果:

```
Input: tensor([-2.,  0.,  2.])
Tanh: tensor([-0.9640,  0.0000,  0.9640])
```

`F.softmax(x, dim)  # probabilities` 将 *logits* 转换为概率（和为 *1*）。用于多分类输出。

示例:

```
import torch
import torch.nn.functional as F
logits = torch.tensor([2.0, 1.0, 0.1])
probs = F.softmax(logits, dim=0)
print(f"Logits: {logits}")
print(f"Softmax: {probs}")
print(f"Sum: {probs.sum():.4f}")
```

运行结果:

```
Logits: tensor([2.0000, 1.0000, 0.1000])
Softmax: tensor([0.6590, 0.2424, 0.0986])
Sum: 1.0000
```

`F.log_softmax(x, dim)` *softmax* 的对数。数值更稳定。与 *NLLLoss* 一起使用。

示例:

```
import torch
import torch.nn.functional as F
logits = torch.tensor([2.0, 1.0, 0.1])
log_probs = F.log_softmax(logits, dim=0)
print(f"Logits: {logits}")
print(f"Log softmax: {log_probs}")
```

运行结果:

```
Logits: tensor([2.0000, 1.0000, 0.1000])
Log softmax: tensor([-0.4170, -1.4170, -2.3170])
```

正则化技术防止训练时过拟合。

F.dropout(x, p, training) 以概率 *p* 随机将元素置为 *0*。仅在训练时有效。

示例:

```
import torch
import torch.nn.functional as F
torch.manual_seed(42)
x = torch.ones(10)
dropped = F.dropout(x, p=0.5, training=True)
print(f"Original: {x}")
print(f"Dropout (p=0.5): {dropped}")
```

运行结果:

```
Original: tensor([1., 1., 1., 1., 1., 1., 1., 1., 1., 1.])
Dropout (p=0.5): tensor([2., 2., 2., 2., 0., 2., 0., 0., 2., 2.])
```

F.batch_norm(x, ...) # normalize 沿批量维度归一化。稳定训练。

示例:

```
import torch
import torch.nn.functional as F
x = torch.randn(2, 3, 4, 4)  # batch, channels, H, W
mean = torch.zeros(3)
var = torch.ones(3)
result = F.batch_norm(x, mean, var)
print(f"Input shape: {x.shape}")
print(f"After batch_norm: mean~{result.mean():.4f}, std~{result.std():.4f}")
```

运行结果:

```
Input shape: torch.Size([2, 3, 4, 4])
After batch_norm: mean~0.1266, std~0.9259
```

F.layer_norm(x, shape) 沿指定维度归一化。用于 *Transformer*。

示例:

```
import torch
import torch.nn.functional as F
x = torch.randn(2, 3, 4)
result = F.layer_norm(x, [3, 4])
print(f"Input shape: {x.shape}")
print(f"After layer_norm: mean~{result.mean():.4f}, std~{result.std():.4f}")
```

运行结果:

```
Input shape: torch.Size([2, 3, 4])
After layer_norm: mean~0.0000, std~1.0215
```

× 损失函数

F.mse_loss(pred, target) 均方误差。差的平方的均值。用于回归。

示例:

```python
import torch
import torch.nn.functional as F
pred = torch.tensor([1.0, 2.0, 3.0])
target = torch.tensor([1.5, 2.0, 2.5])
loss = F.mse_loss(pred, target)
print(f"Prediction: {pred}")
print(f"Target: {target}")
print(f"MSE Loss: {loss:.4f}")
```

运行结果:

```
Prediction: tensor([1., 2., 3.])
Target: tensor([1.5000, 2.0000, 2.5000])
MSE Loss: 0.1667
```

`F.l1_loss(pred, target)` 平均绝对误差。比 *MSE* 对异常值更鲁棒。

示例:

```python
import torch
import torch.nn.functional as F
pred = torch.tensor([1.0, 2.0, 3.0])
target = torch.tensor([1.5, 2.0, 2.5])
loss = F.l1_loss(pred, target)
print(f"Prediction: {pred}")
print(f"Target: {target}")
print(f"L1 Loss: {loss:.4f}")
```

运行结果:

```
Prediction: tensor([1., 2., 3.])
Target: tensor([1.5000, 2.0000, 2.5000])
L1 Loss: 0.3333
```

`F.cross_entropy(logits, labels)` 结合 *log_softmax* 和 *NLLLoss*。多分类的标准损失。

示例:

```python
import torch
import torch.nn.functional as F
logits = torch.tensor([[2.0, 0.5, 0.1], [0.1, 2.0, 0.5]])
labels = torch.tensor([0, 1])  # Class indices
loss = F.cross_entropy(logits, labels)
print(f"Logits: {logits}")
print(f"Labels: {labels}")
print(f"Cross Entropy Loss: {loss:.4f}")
```

运行结果:

```
Logits: tensor([[2.0000, 0.5000, 0.1000],
        [0.1000, 2.0000, 0.5000]])
Labels: tensor([0, 1])
Cross Entropy Loss: 0.3168
```

`F.nll_loss(log_probs, labels)` 负对数似然。与 *log_softmax* 输出一起使用。

示例:

```python
import torch
import torch.nn.functional as F
log_probs = F.log_softmax(torch.tensor([[2.0, 0.5], [0.5, 2.0]]), dim=1)
labels = torch.tensor([0, 1])
loss = F.nll_loss(log_probs, labels)
print(f"NLL Loss: {loss:.4f}")
```

运行结果:

```
NLL Loss: 0.2014
```

F.binary_cross_entropy(pred, target) 二元交叉熵。用于 *sigmoid* 输出的二分类。

示例:

```python
import torch
import torch.nn.functional as F
pred = torch.tensor([0.8, 0.4, 0.9])  # Probabilities
target = torch.tensor([1.0, 0.0, 1.0])  # Binary labels
loss = F.binary_cross_entropy(pred, target)
print(f"Pred: {pred}")
print(f"Target: {target}")
print(f"BCE Loss: {loss:.4f}")
```

运行结果:

```
Pred: tensor([0.8000, 0.4000, 0.9000])
Target: tensor([1., 0., 1.])
BCE Loss: 0.2798
```

F.kl_div(log_pred, target) *Kullback-Leibler* 散度。测量分布之间的差异。

示例:

```python
import torch
import torch.nn.functional as F
log_pred = F.log_softmax(torch.tensor([0.5, 0.3, 0.2]), dim=0)
target = F.softmax(torch.tensor([0.4, 0.4, 0.2]), dim=0)
loss = F.kl_div(log_pred, target, reduction='sum')
print(f"KL Divergence: {loss:.4f}")
```

运行结果:

```
KL Divergence: 0.0035
```

F.cosine_similarity(x1, x2, dim) 测量向量之间的角度。*1* = 同向，*-1* = 反向。

示例:

```python
import torch
import torch.nn.functional as F
x1 = torch.tensor([[1.0, 0.0, 0.0]])
x2 = torch.tensor([[1.0, 1.0, 0.0]])
sim = F.cosine_similarity(x1, x2)
print(f"x1: {x1}")
print(f"x2: {x2}")
print(f"Cosine similarity: {sim}")
```

运行结果:

```
x1: tensor([[1., 0., 0.]])
x2: tensor([[1., 1., 0.]])
Cosine similarity: tensor([0.7071])
```

F.triplet_margin_loss(...) 学习嵌入，使相似项接近，不同项远离。

示例:

```
import torch
import torch.nn.functional as F
anchor = torch.randn(3, 128)
positive = anchor + 0.1 * torch.randn(3, 128)
negative = torch.randn(3, 128)
loss = F.triplet_margin_loss(anchor, positive, negative, margin=1.0)
print(f"Triplet loss: {loss:.4f}")
```

运行结果:

```
Triplet loss: 0.0000
```

池化和卷积

F.max_pool2d(x, kernel_size) 取每个窗口的最大值进行下采样。减少空间维度。

示例:

```
import torch
import torch.nn.functional as F
x = torch.arange(16).float().view(1, 1, 4, 4)
print("Input (4x4):")
print(x.squeeze())
result = F.max_pool2d(x, kernel_size=2)
print("MaxPool2d (kernel=2):")
print(result.squeeze())
```

运行结果:

```
Input (4x4):
tensor([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  6.,  7.],
        [ 8.,  9., 10., 11.],
        [12., 13., 14., 15.]])
MaxPool2d (kernel=2):
tensor([[ 5.,  7.],
        [13., 15.]])
```

F.avg_pool2d(x, kernel_size) 对每个窗口取平均进行下采样。比 *max pooling* 更平滑。

示例:

```
import torch
import torch.nn.functional as F
x = torch.arange(16).float().view(1, 1, 4, 4)
print("Input (4x4):")
```

```
print(x.squeeze())
result = F.avg_pool2d(x, kernel_size=2)
print("AvgPool2d (kernel=2):")
print(result.squeeze())
```

运行结果:

```
Input (4x4):
tensor([[ 0.,  1.,  2.,  3.],
        [ 4.,  5.,  6.,  7.],
        [ 8.,  9., 10., 11.],
        [12., 13., 14., 15.]])
AvgPool2d (kernel=2):
tensor([[ 2.5000,  4.5000],
        [10.5000, 12.5000]])
```

F.adaptive_max_pool2d(x, output_size) 池化到精确的输出尺寸，不受输入大小影响。

示例:

```
import torch
import torch.nn.functional as F
x = torch.randn(1, 1, 7, 7)
result = F.adaptive_max_pool2d(x, output_size=(2, 2))
print(f"Input: {x.shape} -> Output: {result.shape}")
```

运行结果:

```
Input: torch.Size([1, 1, 7, 7]) -> Output: torch.Size([1, 1, 2, 2])
```

F.conv2d(x, weight, bias) *2D* 卷积。*CNN* 的核心操作。提取空间特征。

示例:

```
import torch
import torch.nn.functional as F
x = torch.randn(1, 1, 5, 5)  # batch, in_ch, H, W
weight = torch.randn(1, 1, 3, 3)  # out_ch, in_ch, kH, kW
result = F.conv2d(x, weight)
print(f"Input: {x.shape}")
print(f"Weight: {weight.shape}")
print(f"Output: {result.shape}")
```

运行结果:

```
Input: torch.Size([1, 1, 5, 5])
Weight: torch.Size([1, 1, 3, 3])
Output: torch.Size([1, 1, 3, 3])
```

F.conv_transpose2d(x, weight) 转置卷积（反卷积）。上采样空间维度。

示例:

```
import torch
import torch.nn.functional as F
x = torch.randn(1, 1, 3, 3)
weight = torch.randn(1, 1, 3, 3)
```

```
result = F.conv_transpose2d(x, weight)
print(f"Input: {x.shape}")
print(f"Output: {result.shape}")
```

运行结果:

```
Input: torch.Size([1, 1, 3, 3])
Output: torch.Size([1, 1, 5, 5])
```

F.interpolate(x, size, mode) 使用插值调整张量大小。模式：*nearest*、*bilinear*、*bicubic*。

示例:

```
import torch
import torch.nn.functional as F
x = torch.randn(1, 1, 4, 4)
result = F.interpolate(x, size=(8, 8), mode='bilinear', align_corners=False)
print(f"Input: {x.shape} -> Output: {result.shape}")
```

运行结果:

```
Input: torch.Size([1, 1, 4, 4]) -> Output: torch.Size([1, 1, 8, 8])
```

F.pad(x, pad, mode) 在张量周围添加填充。用于卷积。模式：*constant*、*reflect*、*replicate*。

示例:

```
import torch
import torch.nn.functional as F
x = torch.arange(4).float().view(1, 1, 2, 2)
print("Input:")
print(x.squeeze())
padded = F.pad(x, (1, 1, 1, 1), mode='constant', value=0)
print("Padded (1 on each side):")
print(padded.squeeze())
```

运行结果:

```
Input:
tensor([[0., 1.],
        [2., 3.]])
Padded (1 on each side):
tensor([[0., 0., 0., 0.],
        [0., 0., 1., 0.],
        [0., 2., 3., 0.],
        [0., 0., 0., 0.]])
```

∞ 高级操作

torch.einsum('ij,jk->ik', a, b) 爱因斯坦求和约定。许多张量操作的灵活符号。

示例:

```
import torch
a = torch.randn(2, 3)
b = torch.randn(3, 4)
```

```
result = torch.einsum('ij,jk->ik', a, b)
print(f"einsum('ij,jk->ik'): {a.shape} x {b.shape} = {result.shape}")
# Verify it's matrix multiplication
print(f"Same as matmul: {torch.allclose(result, a @ b)}")
```

运行结果:

```
einsum('ij,jk->ik'): torch.Size([2, 3]) x torch.Size([3, 4]) = torch.Size([2, 4])
Same as matmul: True
```

`torch.topk(x, k, dim)  # top k values` 返回 k 个最大值和索引。比完整排序快。

示例:

```
import torch
x = torch.tensor([1, 5, 3, 9, 2, 7])
vals, idxs = torch.topk(x, k=3)
print(f"Input: {x}")
print(f"Top 3 values: {vals}")
print(f"Top 3 indices: {idxs}")
```

运行结果:

```
Input: tensor([1, 5, 3, 9, 2, 7])
Top 3 values: tensor([9, 7, 5])
Top 3 indices: tensor([3, 5, 1])
```

`torch.sort(x, dim)  # sorted values` 沿维度排序张量。返回值和原始索引。

示例:

```
import torch
x = torch.tensor([3, 1, 4, 1, 5, 9])
vals, idxs = torch.sort(x)
print(f"Original: {x}")
print(f"Sorted: {vals}")
print(f"Indices: {idxs}")
```

运行结果:

```
Original: tensor([3, 1, 4, 1, 5, 9])
Sorted: tensor([1, 1, 3, 4, 5, 9])
Indices: tensor([1, 3, 0, 2, 4, 5])
```

`torch.argsort(x, dim)  # sort indices` 返回排序张量的索引。

示例:

```
import torch
x = torch.tensor([3, 1, 4, 1, 5])
idxs = torch.argsort(x)
print(f"Original: {x}")
print(f"Argsort: {idxs}")
print(f"Sorted via indices: {x[idxs]}")
```

运行结果:

```
Original: tensor([3, 1, 4, 1, 5])
Argsort: tensor([1, 3, 0, 2, 4])
Sorted via indices: tensor([1, 1, 3, 4, 5])
```

`torch.unique(x)  # unique values` 返回唯一元素。可选 *return_counts* 获取频率。

示例:

```
import torch
x = torch.tensor([1, 2, 2, 3, 1, 3, 3, 4])
unique = torch.unique(x)
print(f"Original: {x}")
print(f"Unique: {unique}")
```

运行结果:

```
Original: tensor([1, 2, 2, 3, 1, 3, 3, 4])
Unique: tensor([1, 2, 3, 4])
```

`torch.cat([t1, t2], dim)  # concat` 沿现有维度连接张量。除 *dim* 外形状必须相同。

示例:

```
import torch
a = torch.tensor([[1, 2], [3, 4]])
b = torch.tensor([[5, 6]])
result = torch.cat([a, b], dim=0)
print("Concatenated along dim 0:")
print(result)
```

运行结果:

```
Concatenated along dim 0:
tensor([[1, 2],
        [3, 4],
        [5, 6]])
```

`torch.stack([t1, t2], dim)  # new dim` 沿新维度堆叠张量。所有张量必须形状相同。

示例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
result = torch.stack([a, b], dim=0)
print("Stacked (creates new dim):")
print(result)
print(f"Shape: {result.shape}")
```

运行结果:

```
Stacked (creates new dim):
tensor([[1, 2, 3],
        [4, 5, 6]])
Shape: torch.Size([2, 3])
```

torch.split(x, size, dim) # split 将张量分割为指定大小的块。最后一块可能较小。

示例:

```
import torch
x = torch.arange(10)
splits = torch.split(x, 3)
print(f"Original: {x}")
print("Split into chunks of 3:")
for i, s in enumerate(splits):
    print(f"  {i}: {s}")
```

运行结果:

```
Original: tensor([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
Split into chunks of 3:
  0: tensor([0, 1, 2])
  1: tensor([3, 4, 5])
  2: tensor([6, 7, 8])
  3: tensor([9])
```

torch.chunk(x, chunks, dim) # chunks 将张量分割为指定数量的块。

示例:

```
import torch
x = torch.arange(12)
chunks = torch.chunk(x, 4)
print(f"Original: {x}")
print("Split into 4 chunks:")
for i, c in enumerate(chunks):
    print(f"  {i}: {c}")
```

运行结果:

```
Original: tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
Split into 4 chunks:
  0: tensor([0, 1, 2])
  1: tensor([3, 4, 5])
  2: tensor([6, 7, 8])
  3: tensor([ 9, 10, 11])
```

torch.broadcast_to(x, shape) 显式将张量广播到新形状。不复制数据。

示例:

```
import torch
x = torch.tensor([1, 2, 3])
result = torch.broadcast_to(x, (3, 3))
print(f"Original: {x}")
print("Broadcast to (3, 3):")
print(result)
```

运行结果:

```
Original: tensor([1, 2, 3])
Broadcast to (3, 3):
tensor([[1, 2, 3],
```

```
        [1, 2, 3],
        [1, 2, 3]])
```

torch.flatten(x, start, end) 展平张量。可选起始/结束维度用于部分展平。

示例:

```
import torch
x = torch.randn(2, 3, 4)
flat = torch.flatten(x)
partial = torch.flatten(x, start_dim=1)
print(f"Original: {x.shape}")
print(f"Fully flat: {flat.shape}")
print(f"Flatten from dim 1: {partial.shape}")
```

运行结果:

```
Original: torch.Size([2, 3, 4])
Fully flat: torch.Size([24])
Flatten from dim 1: torch.Size([2, 12])
```

∂ 自动微分

x.requires_grad_(True) 启用张量的梯度跟踪。反向传播所需。

示例:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
print(f"Before: requires_grad = {x.requires_grad}")
x.requires_grad_(True)
print(f"After: requires_grad = {x.requires_grad}")
```

运行结果:

```
Before: requires_grad = False
After: requires_grad = True
```

y.backward() 通过反向传播计算梯度。存储在*.grad* 属性中。

示例:

```
import torch
x = torch.tensor([2.0, 3.0], requires_grad=True)
y = (x ** 2).sum()  # y = x1^2 + x2^2
y.backward()
print(f"x = {x}")
print(f"y = x^2.sum() = {y}")
print(f"dy/dx = {x.grad}")  # 2*x
```

运行结果:

```
x = tensor([2., 3.], requires_grad=True)
y = x^2.sum() = 13.0
dy/dx = tensor([4., 6.])
```

x.grad # gradient *backward()* 后存储累积的梯度。用 *zero_grad()* 重置。

示例:

```
import torch
x = torch.tensor(3.0, requires_grad=True)
y = x ** 2
y.backward()
print(f"x = {x}")
print(f"y = x^2 = {y}")
print(f"dy/dx = {x.grad}")
```

运行结果:

```
x = 3.0
y = x^2 = 9.0
dy/dx = 6.0
```

x.detach() 返回从计算图分离的张量。停止梯度流。

示例:

```
import torch
x = torch.tensor([1.0, 2.0], requires_grad=True)
y = x * 2
z = y.detach()
print(f"y requires_grad: {y.requires_grad}")
print(f"z requires_grad: {z.requires_grad}")
```

运行结果:

```
y requires_grad: True
z requires_grad: False
```

x.clone() 创建数据的副本。修改不影响原始张量。

示例:

```
import torch
x = torch.tensor([1.0, 2.0])
y = x.clone()
y[0] = 99
print(f"Original x: {x}")
print(f"Cloned y: {y}")
```

运行结果:

```
Original x: tensor([1., 2.])
Cloned y: tensor([99.,  2.])
```

with torch.no_grad(): 禁用梯度计算的上下文管理器。用于推理。

示例:

```
import torch
x = torch.tensor([1.0, 2.0], requires_grad=True)
with torch.no_grad():
    y = x * 2
    print(f"Inside no_grad: y.requires_grad = {y.requires_grad}")
```

```
z = x * 2
print(f"Outside: z.requires_grad = {z.requires_grad}")
```

运行结果:

```
Inside no_grad: y.requires_grad = False
Outside: z.requires_grad = True
```

torch.autograd.grad(y, x) 计算梯度而不修改*.grad*。适合二阶导数。

示例:

```
import torch
x = torch.tensor([1.0, 2.0], requires_grad=True)
y = (x ** 3).sum()  # y = x1^3 + x2^3
grad = torch.autograd.grad(y, x)
print(f"x = {x}")
print(f"dy/dx = {grad[0]}")  # 3*x^2
```

运行结果:

```
x = tensor([1., 2.], requires_grad=True)
dy/dx = tensor([ 3., 12.])
```

optimizer.zero_grad() 将梯度重置为 *0*。在每次 *backward* 前调用。

示例:

```
import torch
import torch.optim as optim
x = torch.tensor([1.0], requires_grad=True)
optimizer = optim.SGD([x], lr=0.1)
y = x ** 2
y.backward()
print(f"Grad before zero_grad: {x.grad}")
optimizer.zero_grad()
print(f"Grad after zero_grad: {x.grad}")
```

运行结果:

```
Grad before zero_grad: tensor([2.])
Grad after zero_grad: None
```

optimizer.step() 使用计算的梯度更新参数。*x = x - lr* grad。*

示例:

```
import torch
import torch.optim as optim
x = torch.tensor([5.0], requires_grad=True)
optimizer = optim.SGD([x], lr=0.1)
print(f"Before: x = {x.item():.4f}")
y = x ** 2
y.backward()
optimizer.step()
print(f"After step: x = {x.item():.4f}")
```

运行结果:

```
Before: x = 5.0000
After step: x = 4.0000
```

`torch.nn.utils.clip_grad_norm_(params, max)` 裁剪梯度范数防止梯度爆炸。*RNN* 必需。

示例:

```
import torch
import torch.nn as nn
model = nn.Linear(10, 5)
x = torch.randn(1, 10)
loss = model(x).sum()
loss.backward()
norm_before = sum(p.grad.norm()**2 for p in model.parameters())**0.5
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
norm_after = sum(p.grad.norm()**2 for p in model.parameters())**0.5
print(f"Grad norm before: {norm_before:.4f}")
print(f"Grad norm after clip: {norm_after:.4f}")
```

运行结果:

```
Grad norm before: 6.8840
Grad norm after clip: 1.0000
```

设备操作

`torch.cuda.is_available()` 检查 *CUDA GPU* 是否可用。用于设备选择逻辑。

示例:

```
import torch
print(f"CUDA available: {torch.cuda.is_available()}")
print(f"MPS available: {torch.backends.mps.is_available()}")
```

运行结果:

```
CUDA available: False
MPS available: True
```

`torch.device('cuda'/'cpu'/'mps')` 创建设备对象。与*.to(device)* 一起用于设备放置。

示例:

```
import torch
cpu = torch.device('cpu')
print(f"CPU device: {cpu}")
if torch.cuda.is_available():
    gpu = torch.device('cuda')
    print(f"GPU device: {gpu}")
elif torch.backends.mps.is_available():
    mps = torch.device('mps')
    print(f"MPS device: {mps}")
```

运行结果:

```
CPU device: cpu
MPS device: mps
```

`x.to(device)  # move to device` 将张量移动到指定设备。*GPU* 训练必需。

示例:

```
import torch
x = torch.tensor([1.0, 2.0])
device = torch.device('cpu')
x_dev = x.to(device)
print(f"Tensor on: {x_dev.device}")
```

运行结果:

```
Tensor on: cpu
```

`x.to(dtype)  # change dtype` 将张量转换为不同的数据类型。

示例:

```
import torch
x = torch.tensor([1, 2, 3])
print(f"Original dtype: {x.dtype}")
x_float = x.to(torch.float32)
print(f"After to(float32): {x_float.dtype}")
```

运行结果:

```
Original dtype: torch.int64
After to(float32): torch.float32
```

`x.cuda(), x.cpu()  # shortcuts` 在 *CPU* 和 *CUDA* 之间移动的快捷方法。

示例:

```
import torch
x = torch.tensor([1.0, 2.0])
print(f"Original device: {x.device}")
x_cpu = x.cpu()
print(f"After cpu(): {x_cpu.device}")
```

运行结果:

```
Original device: cpu
After cpu(): cpu
```

`x.device  # check current device` 返回张量存储的设备。

示例:

```
import torch
x = torch.tensor([1.0, 2.0])
print(f"Device: {x.device}")
print(f"Device type: {x.device.type}")
```

运行结果:

```
Device: cpu
Device type: cpu
```

`x.is_cuda  # boolean check` 如果张量在 *CUDA* 设备上则返回 *True*。

示例:

```
import torch
x = torch.tensor([1.0, 2.0])
print(f"is_cuda: {x.is_cuda}")
```

运行结果:

```
is_cuda: False
```

`torch.cuda.empty_cache()` 释放未使用的缓存 *GPU* 内存。对 *OOM* 错误有帮助。

示例:

```
import torch
# Free unused cached memory (only affects CUDA)
if torch.cuda.is_available():
    torch.cuda.empty_cache()
    print("CUDA cache cleared")
else:
    print("CUDA not available - cache clearing skipped")
```

运行结果:

```
CUDA not available - cache clearing skipped
```

`torch.cuda.device_count()` 返回可用的 *CUDA* 设备数量。

示例:

```
import torch
if torch.cuda.is_available():
    count = torch.cuda.device_count()
    print(f"Number of GPUs: {count}")
else:
    print("CUDA not available")
```

运行结果:

```
CUDA not available
```

※ 实用工具

`x.dtype, x.shape, x.size()` 基本张量属性。*shape* 和 *size()* 相同。

示例:

```
import torch
x = torch.randn(2, 3, 4)
print(f"dtype: {x.dtype}")
print(f"shape: {x.shape}")
print(f"size(): {x.size()}")
```

运行结果:

```
dtype: torch.float32
shape: torch.Size([2, 3, 4])
size(): torch.Size([2, 3, 4])
```

`x.numel()  # number of elements` 返回元素总数。所有维度的乘积。

示例:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Shape: {x.shape}")
print(f"Number of elements: {x.numel()}")
```

运行结果:

```
Shape: torch.Size([2, 3, 4])
Number of elements: 24
```

`x.dim()  # number of dimensions` 返回张量的维数（秩）。

示例:

```
import torch
x = torch.randn(2, 3, 4)
print(f"Shape: {x.shape}")
print(f"Dimensions: {x.dim()}")
```

运行结果:

```
Shape: torch.Size([2, 3, 4])
Dimensions: 3
```

`x.ndimension()  # same as dim()` *dim()* 的别名。返回维数。

示例:

```
import torch
x = torch.randn(2, 3, 4)
print(f"ndimension(): {x.ndimension()}")
print(f"Same as dim(): {x.dim()}")
```

运行结果:

```
ndimension(): 3
Same as dim(): 3
```

`x.is_contiguous()` 检查张量在内存中是否连续。*view()* 需要。

示例:

```
import torch
x = torch.randn(3, 4)
y = x.transpose(0, 1)
print(f"Original is_contiguous: {x.is_contiguous()}")
print(f"Transposed is_contiguous: {y.is_contiguous()}")
```

运行结果:

```
Original is_contiguous: True
Transposed is_contiguous: False
```

`x.float(), x.int(), x.long()` *dtype* 转换的快捷方法。

示例:

```
import torch
x = torch.tensor([1, 2, 3])
print(f"Original: {x.dtype}")
print(f"float(): {x.float().dtype}")
print(f"long(): {x.long().dtype}")
```

运行结果:

```
Original: torch.int64
float(): torch.float32
long(): torch.int64
```

`x.half(), x.double() # fp16, fp64` *half* 精度（*16* 位）用于快速训练，*double* 用于高精度。

示例:

```
import torch
x = torch.tensor([1.0, 2.0])
print(f"Original: {x.dtype}")
print(f"half() (fp16): {x.half().dtype}")
print(f"double() (fp64): {x.double().dtype}")
```

运行结果:

```
Original: torch.float32
half() (fp16): torch.float16
double() (fp64): torch.float64
```

`torch.from_numpy(arr)` 将 *NumPy* 数组转换为张量。与原数组共享内存。

示例:

```
import torch
import numpy as np
arr = np.array([1, 2, 3])
x = torch.from_numpy(arr)
print(f"NumPy: {arr}, dtype={arr.dtype}")
print(f"Tensor: {x}, dtype={x.dtype}")
```

运行结果:

```
NumPy: [1 2 3], dtype=int64
Tensor: tensor([1, 2, 3]), dtype=torch.int64
```

`x.numpy() # CPU only` 将张量转换为 *NumPy* 数组。必须在 *CPU* 上。

示例:

```
import torch
x = torch.tensor([1.0, 2.0, 3.0])
arr = x.numpy()
```

```
print(f"Tensor: {x}")
print(f"NumPy: {arr}, dtype={arr.dtype}")
```

运行结果:

```
Tensor: tensor([1., 2., 3.])
NumPy: [1. 2. 3.], dtype=float32
```

torch.save(obj, path) 将张量或模型保存到文件。使用 *pickle* 序列化。

示例:

```
import torch
import os
x = torch.tensor([1, 2, 3])
torch.save(x, '/tmp/tensor.pt')
print(f"Saved tensor to /tmp/tensor.pt")
print(f"File size: {os.path.getsize('/tmp/tensor.pt')} bytes")
```

运行结果:

```
Saved tensor to /tmp/tensor.pt
File size: 1570 bytes
```

torch.load(path) 从文件加载保存的张量或模型。

示例:

```
import torch
torch.save(torch.tensor([1, 2, 3]), '/tmp/tensor.pt')
loaded = torch.load('/tmp/tensor.pt', weights_only=True)
print(f"Loaded: {loaded}")
```

运行结果:

```
Loaded: tensor([1, 2, 3])
```

torch.manual_seed(seed) 设置随机种子以实现可复现性。

示例:

```
import torch
torch.manual_seed(42)
a = torch.rand(3)
torch.manual_seed(42)
b = torch.rand(3)
print(f"First: {a}")
print(f"Second (same seed): {b}")
print(f"Equal: {torch.equal(a, b)}")
```

运行结果:

```
First: tensor([0.8823, 0.9150, 0.3829])
Second (same seed): tensor([0.8823, 0.9150, 0.3829])
Equal: True
```

≈ 比较操作

torch.eq(a, b) or a == b 逐元素相等比较。返回布尔张量。

示例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([1, 0, 3])
print(f"a: {a}")
print(f"b: {b}")
print(f"a == b: {a == b}")
```

运行结果:

```
a: tensor([1, 2, 3])
b: tensor([1, 0, 3])
a == b: tensor([ True, False,  True])
```

torch.ne(a, b) or a != b 逐元素不等比较。

示例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([1, 0, 3])
print(f"a != b: {a != b}")
```

运行结果:

```
a != b: tensor([False,  True, False])
```

torch.gt(a, b) or a > b 逐元素大于比较。

示例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([2, 2, 2])
print(f"a: {a}, b: {b}")
print(f"a > b: {a > b}")
```

运行结果:

```
a: tensor([1, 2, 3]), b: tensor([2, 2, 2])
a > b: tensor([False, False,  True])
```

torch.lt(a, b) or a < b 逐元素小于比较。

示例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([2, 2, 2])
print(f"a < b: {a < b}")
```

运行结果:

```
a < b: tensor([ True, False, False])
```

torch.ge(a, b) or a >= b 逐元素大于等于比较。

示例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([2, 2, 2])
print(f"a >= b: {a >= b}")
```

运行结果:

```
a >= b: tensor([False,  True,  True])
```

torch.le(a, b) or a <= b 逐元素小于等于比较。

示例:

```
import torch
a = torch.tensor([1, 2, 3])
b = torch.tensor([2, 2, 2])
print(f"a <= b: {a <= b}")
```

运行结果:

```
a <= b: tensor([ True,  True, False])
```

torch.allclose(a, b, rtol, atol) 检查所有元素是否在容差范围内接近。用于 *float* 比较。

示例:

```
import torch
a = torch.tensor([1.0, 2.0])
b = torch.tensor([1.0001, 2.0001])
print(f"a: {a}")
print(f"b: {b}")
print(f"allclose (default tol): {torch.allclose(a, b)}")
print(f"allclose (rtol=1e-3): {torch.allclose(a, b, rtol=1e-3)}")
```

运行结果:

```
a: tensor([1., 2.])
b: tensor([1.0001, 2.0001])
allclose (default tol): False
allclose (rtol=1e-3): True
```

torch.isnan(x) 元素为 *NaN*（非数字）处返回 *True*。

示例:

```
import torch
x = torch.tensor([1.0, float('nan'), 3.0])
print(f"x: {x}")
print(f"isnan: {torch.isnan(x)}")
```

运行结果:

```
x: tensor([1., nan, 3.])
isnan: tensor([False,  True, False])
```

torch.isinf(x) 元素为无穷大处返回 *True*。

示例:

```
import torch
x = torch.tensor([1.0, float('inf'), float('-inf')])
print(f"x: {x}")
print(f"isinf: {torch.isinf(x)}")
```

运行结果:

```
x: tensor([1., inf, -inf])
isinf: tensor([False,  True,  True])
```

torch.isfinite(x) 元素为有限（非 *inf* 非 *nan*）处返回 *True*。

示例:

```
import torch
x = torch.tensor([1.0, float('inf'), float('nan')])
print(f"x: {x}")
print(f"isfinite: {torch.isfinite(x)}")
```

运行结果:

```
x: tensor([1., inf, nan])
isfinite: tensor([ True, False, False])
```

● 张量方法

x.T # transpose (2D) *2D* 转置的简写。高维用 *permute*。

示例:

```
import torch
x = torch.tensor([[1, 2, 3], [4, 5, 6]])
print("Original:")
print(x)
print("x.T:")
print(x.T)
```

运行结果:

```
Original:
tensor([[1, 2, 3],
        [4, 5, 6]])
x.T:
tensor([[1, 4],
        [2, 5],
        [3, 6]])
```

x.H # conjugate transpose 复数张量的共轭转置。转置 + 共轭。

示例:

```
import torch
x = torch.tensor([[1+2j, 3+4j], [5+6j, 7+8j]])
print("Original:")
```

```
print(x)
print("x.H (conjugate transpose):")
print(x.H)
```

运行结果:

```
Original:
tensor([[1.+2.j, 3.+4.j],
        [5.+6.j, 7.+8.j]])
x.H (conjugate transpose):
tensor([[1.-2.j, 5.-6.j],
        [3.-4.j, 7.-8.j]])
```

`x.real, x.imag  # complex parts` 访问复数张量的实部和虚部。

示例:

```
import torch
x = torch.tensor([1+2j, 3+4j])
print(f"Complex: {x}")
print(f"Real: {x.real}")
print(f"Imag: {x.imag}")
```

运行结果:

```
Complex: tensor([1.+2.j, 3.+4.j])
Real: tensor([1., 3.])
Imag: tensor([2., 4.])
```

`x.abs(), x.neg()  # absolute, negate` 绝对值和取负的方法。

示例:

```
import torch
x = torch.tensor([-3, -1, 0, 2, 4])
print(f"x: {x}")
print(f"abs: {x.abs()}")
print(f"neg: {x.neg()}")
```

运行结果:

```
x: tensor([-3, -1,  0,  2,  4])
abs: tensor([3, 1, 0, 2, 4])
neg: tensor([ 3,  1,  0, -2, -4])
```

`x.reciprocal(), x.pow(n)` 作为方法的倒数（*1/x*）和幂运算。

示例:

```
import torch
x = torch.tensor([1.0, 2.0, 4.0])
print(f"x: {x}")
print(f"reciprocal: {x.reciprocal()}")
print(f"pow(2): {x.pow(2)}")
```

运行结果:

```
x: tensor([1., 2., 4.])
reciprocal: tensor([1.0000, 0.5000, 0.2500])
pow(2): tensor([ 1.,  4., 16.])
```

`x.sqrt()`, `x.exp()`, `x.log()` 作为张量方法的常见数学运算。

示例:

```python
import torch
x = torch.tensor([1.0, 4.0, 9.0])
print(f"x: {x}")
print(f"sqrt: {x.sqrt()}")
print(f"exp: {torch.tensor([0.0, 1.0]).exp()}")
print(f"log: {torch.tensor([1.0, 2.718]).log()}")
```

运行结果:

```
x: tensor([1., 4., 9.])
sqrt: tensor([1., 2., 3.])
exp: tensor([1.0000, 2.7183])
log: tensor([0.0000, 0.9999])
```

`x.item()  # get scalar value` 从单元素张量提取标量值作为 *Python* 数字。

示例:

```python
import torch
x = torch.tensor(3.14159)
val = x.item()
print(f"Tensor: {x}")
print(f"Python float: {val}")
print(f"Type: {type(val)}")
```

运行结果:

```
Tensor: 3.141590118408203
Python float: 3.141590118408203
Type: <class 'float'>
```

`x.tolist()  # to Python list` 将张量转换为嵌套的 *Python* 列表。

示例:

```python
import torch
x = torch.tensor([[1, 2], [3, 4]])
lst = x.tolist()
print(f"Tensor: {x}")
print(f"List: {lst}")
print(f"Type: {type(lst)}")
```

运行结果:

```
Tensor: tensor([[1, 2],
        [3, 4]])
List: [[1, 2], [3, 4]]
Type: <class 'list'>
```

`x.all(), x.any()  # boolean checks` 检查所有或任一元素是否为 *True*。

示例:

```
import torch
x = torch.tensor([True, True, False])
print(f"x: {x}")
print(f"all: {x.all()}")
print(f"any: {x.any()}")
```

运行结果:

```
x: tensor([ True,  True, False])
all: False
any: True
```

`x.nonzero()  # non-zero indices` 返回非零元素的索引。

示例:

```
import torch
x = torch.tensor([0, 1, 0, 2, 0, 3])
indices = x.nonzero()
print(f"x: {x}")
print(f"Non-zero indices: {indices.squeeze()}")
```

运行结果:

```
x: tensor([0, 1, 0, 2, 0, 3])
Non-zero indices: tensor([1, 3, 5])
```

`x.fill_(val), x.zero_()  # in-place` 原地填充值或零。下划线后缀 = 原地操作。

示例:

```
import torch
x = torch.empty(2, 3)
x.fill_(5.0)
print("After fill_(5.0):")
print(x)
x.zero_()
print("After zero_():")
print(x)
```

运行结果:

```
After fill_(5.0):
tensor([[5., 5., 5.],
        [5., 5., 5.]])
After zero_():
tensor([[0., 0., 0.],
        [0., 0., 0.]])
```

`x.normal_(), x.uniform_()  # random` 原地随机初始化。适合权重初始化。

示例:

```python
import torch
torch.manual_seed(42)
x = torch.empty(2, 3)
x.normal_(mean=0, std=1)
print("After normal_(0, 1):")
print(x)
x.uniform_(0, 1)
print("After uniform_(0, 1):")
print(x)
```

运行结果:

```
After normal_(0, 1):
tensor([[ 0.3367,  0.1288,  0.2345],
        [ 0.2303, -1.1229, -0.1863]])
After uniform_(0, 1):
tensor([[0.8694, 0.5677, 0.7411],
        [0.4294, 0.8854, 0.5739]])
```

`x.add_(y), x.mul_(y)  # in-place ops` 原地算术。直接修改张量节省内存。

示例:

```python
import torch
x = torch.tensor([1.0, 2.0, 3.0])
print(f"Original: {x}")
x.add_(10)
print(f"After add_(10): {x}")
x.mul_(2)
print(f"After mul_(2): {x}")
```

运行结果:

```
Original: tensor([1., 2., 3.])
After add_(10): tensor([11., 12., 13.])
After mul_(2): tensor([22., 24., 26.])
```